

ДИСЦИПЛИНА	Технологическая (проектно-технологическая) практика
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания
ПРЕПОДАВАТЕЛЬ	Астафьев Рустам Уралович
СЕМЕСТР	1 семестр, 2025/2026 уч. год

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ	3
ПЕРЕЧЕНЬ ВЫПОЛНЕННЫХ РАБОТ	4
ЗАКЛЮЧЕНИЕ	13
<i>ПРИЛОЖЕНИЕ А</i>	14

ВВЕДЕНИЕ

Современный мир ИТ направлен на создание сверхсложных систем, обработку огромных массивов данных и обеспечение того, чтобы пользователи могли легко работать со всеми различными услугами с помощью цифровой обработки. Технология, которую мы создали в НИИАС, была частью основного плана с 3 октября 2024 года. Задания были предоставлены Юсиповым Ренатом Алиевичем. Получение опыта практической работы по разработке корпоративной программы, адаптированной к потребностям бизнеса, особенно для визуализации и управления железнодорожной инфраструктурой на интерактивной карте.

Целью практики являлась разработка интерактивного веб-модуля для визуализации и управления объектами железнодорожной инфраструктуры, способного стабильно и плавно работать при отображении большого объема данных, в том числе на очень слабых и древних машинах.

Внимание уделялось обеспечению производительности в визуализации большого числа объектов, правильной обработке событий пользователей (ввод, щелчки, драгги), поддержке масштаба и панорамы, а также архитектурной организации решений (слоев, состав приложений, Результатом этой работы стал единый инструмент управления визуальной инфраструктурой в графической форме.

ПЕРЕЧЕНЬ ВЫПОЛНЕННЫХ РАБОТ

Во время практики я завершил проект по созданию интерактивной карты российской железнодорожной сети. В отношении структуры отчета проект был разделен на следующие последовательные этапы: аналитика и моделирование, реализация функции рисования, добавление интерактивности, реализация слоев и механизмов редактирования, а также этап оптимизации и интеграции. Этот подход обеспечивает отслеживаемость результатов и отражает типичную организацию работы в инженерных проектах.

1. Анализ требований и проектирование интерактивной карты железнодорожной сети РФ

Дата выдачи: 3 октября 2025 г.

Автор задачи: Юсипов Ренат Алиевич (руководитель работ).

Срок для исполнения: 10 рабочих дней.

Постановка задачи заключалась в создании интерактивной карты железнодорожной сети Российской Федерации, предназначенной для визуализации объектов инфраструктуры и выполнения действий управления на уровне графического представления. Карта должна была работать в браузере, обеспечивать высокую производительность при большом количестве отображаемых объектов, а также иметь удобные механизмы взаимодействия пользователя с элементами карты.

На этапе анализа были уточнены ключевые бизнес-сценарии использования карты. В частности, система должна была обеспечивать отображение перегонов узлов подписей опорных и контрольных точек, а также поддержку слоёв с возможностью включения и отключения групп объектов. Кроме того, требовалась реализация масштабирования и панорамирования карты с плавной навигацией обработка событий наведения и кликов по объектам, а также наличие режима редактирования, предусматривающего перемещение точек методом Drag'n'Drop с фиксацией внесённых изменений.

Далее была сформирована техническая концепция решения. Для отрисовки было принято использовать Canvas API как более производительный вариант по сравнению с DOM/SVG при количестве объектов порядка нескольких тысяч. Клиентская часть проектировалась на React + TypeScript, при этом состояние приложения (активные слои, параметры камеры, выбранный объект, режимы редактирования) планировалось хранить в Redux Toolkit. Для серверной части предполагалось использование C# .NET 5, предоставляющего API для загрузки инфраструктурных данных и сохранения результатов редактирования.

На основании требований была сделана архитектура решения. В основе лежит единая модель данных, описывающая объекты инфраструктуры с использованием унифицированных структур, включающих идентификатор объекта, его тип, геометрию, параметры отображения и вспомогательные метаданные

Для корректного отображения и взаимодействия с картой была реализована система координат с разделением мировых и экранных координат. Преобразования масштабирования и смещения (scale и translate) позволили обеспечить поддержку плавного масштабирования и панорамирования без потери точности.

Отображение объектов было сделано с использованием системы слоёв, представляющих собой независимые наборы графических примитивов. Это дало возможность управлять видимостью отдельных групп объектов и выполнять выборочную перерисовку только тех элементов, которые требуют обновления, что положительно сказалось на производительности.

Интерактивное взаимодействие с картой реализовано через единый механизм обработки пользовательских событий. Для определения объекта под курсором был разработан модуль hit-testing, обеспечивающий корректную обработку наведения и кликов даже при большом количестве отображаемых элементов.

Для взаимодействия с серверной частью был определён API, предназначенный для получения данных и передачи изменений. В рамках интеграции была предусмотрена обработка возможных ошибок и контроль целостности передаваемых данных.

Фактический срок реализации: 5 рабочих дней.

Комментарий принимающего лица:

«Проектирование выполнено с учётом требований к производительности и расширяемости. Предложенная архитектура обеспечивает удобство последующей доработки и поддерживает ключевые сценарии использования».

2. Реализация отрисовки более 5000 объектов железнодорожной инфраструктуры на Canvas

Дата выдачи: 13 октября 2025 г.

Автор задачи: Юсипов Ренат Алиевич (руководитель работ).

Срок для исполнения: 15 рабочих дней.

На данном этапе была реализована подсистема визуализации объектов на Canvas. Требование заключалось в отображении более 5000 графических элементов (перегоны, подписи, узлы и др.) с сохранением отзывчивости интерфейса. Отдельное внимание уделялось тому, чтобы перерисовка оставалась быстрой при масштабировании и панорамировании, а также при переключении слоёв.

Реализация отрисовки основывалась на использовании пакетного подхода, при котором формировался список команд отрисовки с учётом активных слоёв и текущего масштаба. Для снижения вычислительной нагрузки были вынесены повторяющиеся операции и организовано повторное использование ранее рассчитанных геометрий. С целью повышения производительности из процесса отрисовки исключались объекты, находящиеся вне видимой области, на основе проверки по ограничивающим прямоугольникам. Дополнительно была выполнена адаптация под HiDPI-экраны с учётом значения `devicePixelRatio`, что обеспечило чёткое отображение линий и текста. Перерисовка сцены управлялась

через единый цикл рендера, что позволило избежать лишних обновлений и сохранить плавность работы интерфейса.

Для каждого типа объекта были определены правила визуализации (цвета, толщины линий, маркеры, отображение подписей в зависимости от масштаба). Это обеспечило читаемость карты при различных уровнях увеличения: от обзорного режима до детализированного представления участка инфраструктуры.

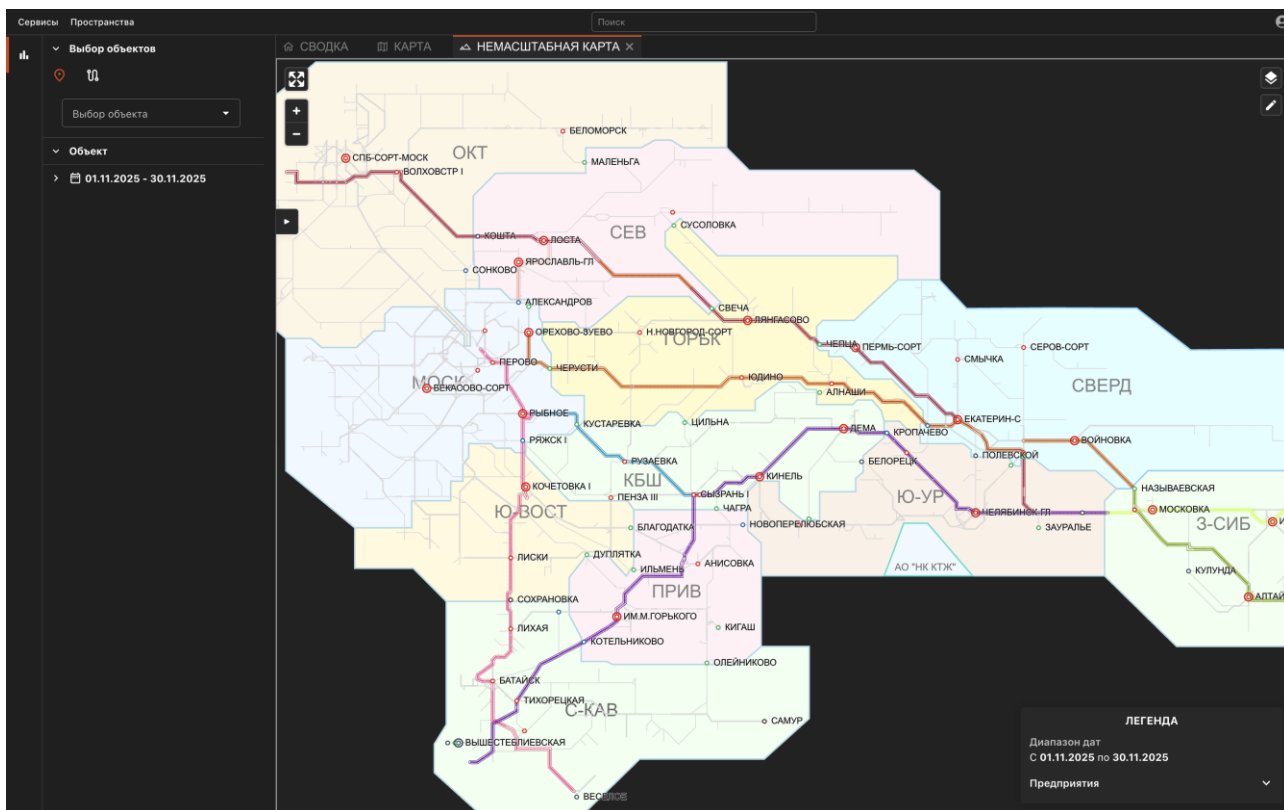


Рисунок 2.1 – Визуализации объектов инфраструктуры

Фактический срок реализации: 9 рабочих дней.

Комментарий принимающего лица:

«Подсистема отрисовки демонстрирует устойчивую производительность при большом количестве объектов. Реализация соответствует ожиданиям и обеспечивает основу для интерактивных функций».

3. Реализация интерактивности (zoom/pan/hover/click) и hit-testing

Дата выдачи: 28 октября 2024 г.

Автор задачи: Юсипов Ренат Алиевич (руководитель работ).

Срок для исполнения: 14 рабочих дней.

На данном этапе была реализована интерактивная модель взаимодействия с картой: масштабирование и панорамирование, а также обработка событий наведения и кликов по объектам. Существенная сложность заключалась в корректном определении объекта под курсором при большом количестве элементов и активных преобразованиях координат (масштаб/смещение).

Были разработаны основные механизмы работы с картой. Масштабирование осуществлялось с помощью колеса мыши с привязкой к позиции курсора, что обеспечивало удобный контроль над областью просмотра, при этом были заданы ограничения на минимальный и максимальный уровни масштаба. Панорамирование сцены выполнялось с использованием мыши и тач-жестов с учётом допустимых границ перемещения. При наведении курсора на объекты реализовывалась их визуальная подсветка и отображение краткой информации, а при клике происходил выбор объекта с фиксацией состояния в хранилище Redux Toolkit и выводом расширенных данных в пользовательском интерфейсе.

Для проверки попаданий мы создали специальный блок для согласования координат. Позиция курсора менялась с экранного вида на мировой, используя обратную трансформацию, а затем среди действующих слоёв искали соответствующие элементы. Чтобы уменьшить число сравнений, сначала применялись базовые пространственные критерии и отсеивание по виду элемента. Когда этого не хватало, мы переходили к более детальной проверке, например, вычисляли удалённость до отрезка для прямых объектов или смотрели, попадает ли точка в круг для точек.

Такой подход дал возможность нормально работать с картой даже когда объектов очень много. Человек может без проблем двигаться по всей карте и быстро находить нужные вещи без ощутимых пауз в обычных ситуациях.

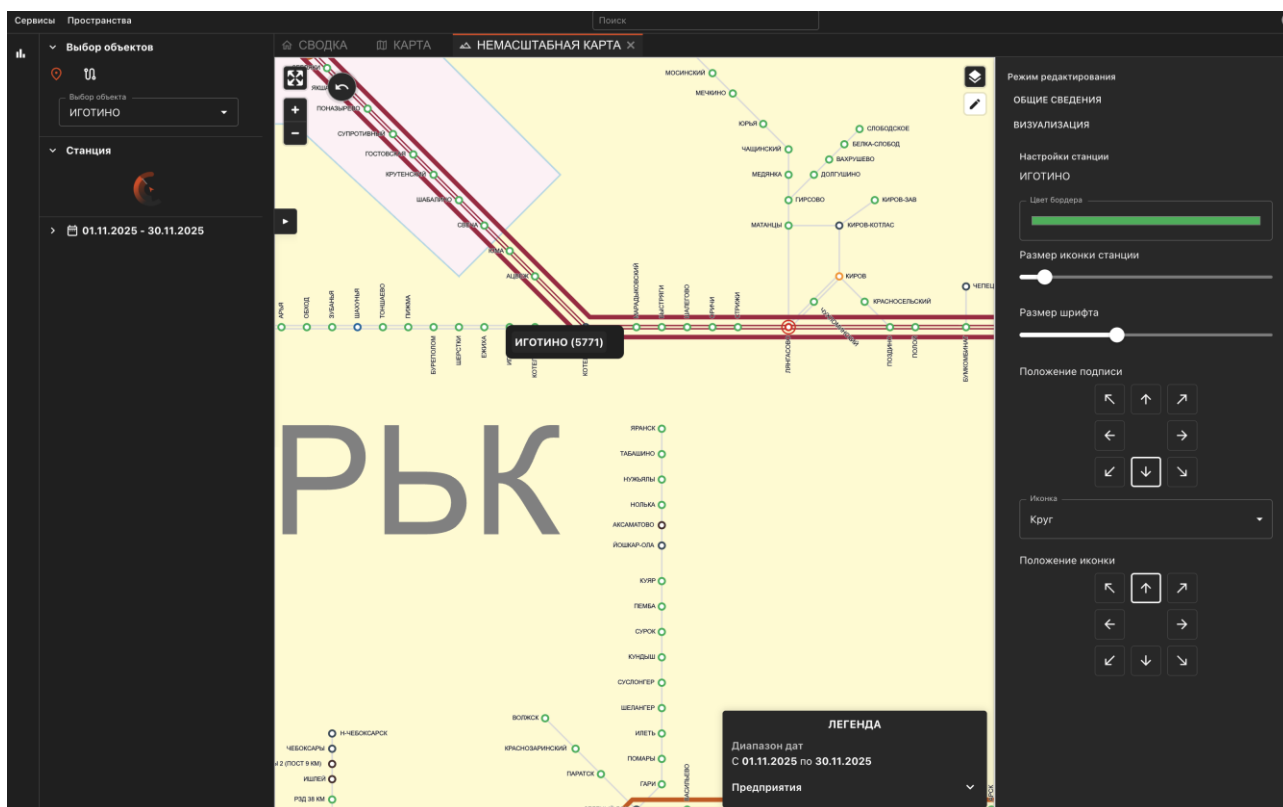


Рисунок 3.1 – Интерактивное взаимодействие с объектами карты

Фактический срок реализации: 6 рабочих дней.

Комментарий принимающего лица:

«Интерактивные функции реализованы корректно: навигация плавная, выбор объектов удобный. Модуль определения объектов под курсором работает стабильно и обеспечивает требуемую точность».

4. Реализация системы слоёв и Drag'n'Drop редактирования объектов

Дата выдачи: 12 ноября 2025 г.

Автор задачи: Юсипов Ренат Алиевич (руководитель работ).

Срок для исполнения: 8 рабочих дней.

Цель этапа заключалась в расширении функциональности карты: добавлении системы слоёв (переключаемых наборов объектов) и реализации режима редактирования с использованием Drag'n'Drop перемещения точек на карте. Данный функционал был необходим для перехода от “просмотра” к “управлению” инфраструктурой в графическом виде.

Система слоёв была реализована в виде набора независимых конфигураций отображения, управление которыми осуществлялось через состояние приложения с использованием Redux Toolkit. Такой подход позволил пользователю включать и отключать отображение отдельных типов объектов, включая подписи, узлы и вспомогательные точки, а также переключаться между преднастроенными профилями отображения в зависимости от текущего сценария работы. При необходимости реализована возможность сохранения выбранных настроек с последующим быстрым возвратом к ним.

Для режима редактирования реализован Dragn Drop: при выборе точки пользователь может перемещать её по карте, при этом координаты обновляются в мировом пространстве с учётом текущих преобразований. В процессе перемещения обеспечивалась визуальная обратная связь (подсветка, отображение текущих координат/привязок при необходимости). После завершения перемещения формировалось изменение, которое передавалось на сервер для сохранения.

Дополнительно были предусмотрены базовые проверки приватности: блокировка редактирования в режиме просмотра, контроль прав, обработка отмены действия и предотвращение случайных перемещений.

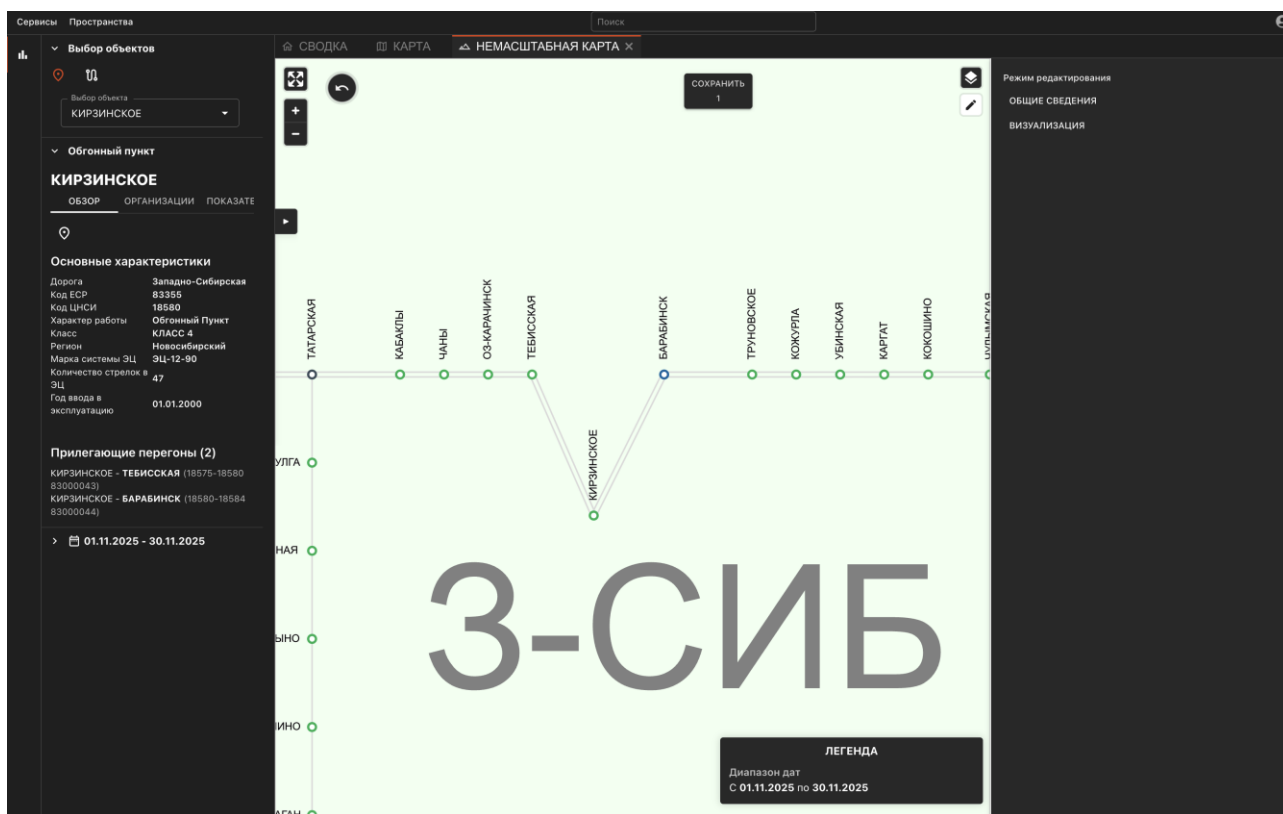


Рисунок 4.1 – Работа слоёв и перемещения точек на карте

Фактический срок реализации: 7 рабочих дней.

Комментарий принимающего лица:

«Функциональность слоёв и редактирования существенно повысила прикладную ценность решения. Drag'n'Drop реализован корректно, изменения фиксируются надёжно и удобны для пользователей».

5. Оптимизация производительности и интеграция решения с backend (C# .NET 5)

Дата выдачи: 28 ноября 2025 г.

Автор задачи: Юсипов Ренат Алиевич (руководитель работ).

Срок для исполнения: 6 рабочих дней.

Заключительный этап был посвящён оптимизации производительности и обеспечению устойчивой интеграции с серверной частью. При интенсивной работе с картой (частые перемещения, переключения слоёв, редактирование) требовалось минимизировать количество лишних перерисовок и сетевых запросов, а также обеспечить стабильность передачи изменений на сервер.

В последнем этапе работы была проведена оптимизация обновлений состояния приложения, включающая устранение избыточных вызовов `dispatch` и перерасчётов, а также нормализацию структур данных в `Redux Toolkit`. Для повышения производительности отрисовки была сокращена частота перерисовок `Canvas` за счёт перехода к модели обновления по необходимости и уменьшения объёма вычислений в обработчиках событий. Загрузка данных была упорядочена посредством пакетной и постраничной передачи информации с использованием кеширования результатов запросов. Также была реализована интеграция механизма сохранения изменений, включающая формирование патчей, валидацию данных, обработку ошибок серверных ответов и повторную отправку запросов при временных сбоях. Дополнительно выполнено согласование API-контрактов с серверной частью, реализованной на платформе `C# .NET 5`, и проведено тестирование типовых сценариев работы системы.

После оптимизаций карта сохраняла плавность при навигации и корректно обрабатывала интерактивные операции при большом количестве объектов. Интеграция с серверной частью позволила использовать решение как единый инструмент для работы с инфраструктурными данными: отображение, выбор, анализ и редактирование в графическом представлении.

Фактический срок реализации: 5 рабочих дней.

Комментарий принимающего лица:

«В ходе ревью проблем в работе не выявлено. Решение стабильно работает в основных сценариях и корректно отрабатывает с серверной частью.»

ЗАКЛЮЧЕНИЕ

В ходе прохождения технологической практики в ОАО «НИИАС» была выполнена комплексная разработка интерактивной карты железнодорожной сети Российской Федерации, включающей визуализацию более 5000 объектов, поддержку масштабирования и панорамирования, интерактивные события переключаемые слои, а также режим редактирования с использованием Drag и Drop перемещения точек.

Практика позволила закрепить и расширить навыки разработки корпоративных веб-приложений. Были проапгрейжены знания по стеку TypeScript, React, Redux Toolkit и Canvas API, а также получен опыт интеграции клиентской части с серверной подсистемой на C# .NET 5. Внимание уделялось архитектуре, производительности и удобству работы пользователей с визуальными данными.

Результатом прохождения практики стала реализация и внедрение интерактивного инструмента графического представления железнодорожной инфраструктуры, предназначенного для анализа и управления объектами сети в визуальной форме.

В ходе работы были решены практические задачи, связанные с отображением большого количества объектов, обеспечением плавной навигации по карте, а также поддержкой редактирования и актуализации данных. Разработанное упростило решение выполнение операций, связанных с анализом состояния инфраструктуры, и повысило удобство работы пользователей с визуальными данными.

Опыт полученный во время практики, пригодится при дальнейшей работе над пользовательскими интерфейсами, связанными с визуализацией данных.

ПРИЛОЖЕНИЕ А

Пример фрагмента TypeScript-кода

Ниже приведён фрагмент кода, иллюстрирующий подход к преобразованию координат и обработке событий при интерактивной работе с canvas.

Листинг А.1 — Фрагмент кода отрисовки canvas

```
import React, { useRef, useEffect, useState, useMemo, useCallback,
forwardRef, useImperativeHandle } from 'react';
import { Box } from '@mui/material';
import {
  drawGrid,
  drawShapes,
  drawCustomEdges,
  drawNodeLabels,
  drawFixedSizeStations,
  strokeFillText
} from './utils/drawing.ts';
import {
  INITIAL_ZOOM,
  WORLD_SCALE,
  MIN_NODE_SCREEN_FACTOR,
  NODE_RADIUS_WORLD,
  SCALE_POWER,
  ICON_GAP,
  MIN_ZOOM,
  MAX_ZOOM,
  ALL_ORG_GROUP_IDS,
  DARK_SETTINGS,
  LIGHT_SETTINGS,
  RING_GAP_PX,
  RING_THICKNESS_PX,
  RING_GLOW_PX,
  MIN_RING_RADIUS_PX,
  COLOR,
  ICON_TYPE_TO_NUM,
  DASHBOARD_MAP_INSTANCE,
  NUM_TO_ICON_POS8,
  ICON_POS8_TO_NUM
} from './utils/constants';
import type { StationSettingsState } from
'./components/StationSidebar';
import Fullscreen from './assets/Fullscreen.svg';
import { fitToPoints, computeGridOffset, snapPoint, quantize }
from './utils/layout';
import { geojsonDashboardAPI, stationsAPI, documentService,
```

```

tractionAPI, routesAPI } from 'services';
import LoaderRing from 'shared/ui/LoaderRing';
import { Network } from 'vis-network';
import { DataSet, DataView } from 'vis-data';

```

Продолжение листинга А.1

```

import HoverPopupControl from './HoverPopupControl';
import 'maplibre-gl/dist/maplibre-gl.css';
import { drawAreasPolygons, drawAreaHandles, findAreaAt,
findHandleNear, Area } from './utils/areas';
import { drawTextObjects, getHarColor, createCircleSvgDataUrl,
iconPxAt } from './utils/helpers.ts';
import {
    setFocusedObject,
    setSelectedSteps,
    setSelectedPeregonsIds,
    setSelectionStartStationId,
    setSelectionEndStationId,
    setSelectedWarrantyAreaId,
    setSelectedLocomotiveCirculationAreaId,
    setSelectedLocomotiveWorkTeamAreaId,
    setIsInSelectionMode
} from 'store/reducers/src/AppSlice.ts';
import { useAppDispatch, useAppSelector } from 'hooks';
import { useGeojsonDashboardSettings } from
'./settings/useGeojsonDashboardSettings';
import { useTheme } from '@mui/material/styles';
import PoleSvg from './assets/Столб.svg';
import Layers from './assets/layers.svg';
import AnchorSvg from './assets/anchor-svgrepo-com.svg';
import { AnyArea, IStationGeo } from 'types';
import { RootState } from 'store';
import {
    IDragAction,
    AnchorEl,
    NodeUpdate,
    IconType,
    MirrorEvent,
    GeojsonDashboardProps,
    GeojsonDashboardHandle,
    MirrorToggleKey,
    Id,
    RawEdge,
    VisNode,
    VisEdge,
    Position,
    HoverPopupData,
    ClickEventParams,
    Organisation,
    PeregonRow,
    PeregonProps,
    StationsResponse,
    makeStationGeo,

```

```
    FocusedObjectLocal,  
    NodeUpdatePayload,  
    IndicatorVisual,  
    pickFromMap,
```

Продолжение листинга А.1

```
    rgbToCss,  
    isStationDtoLikeArray,  
    VisDragParams,  
    VisClickParams,  
    PeregonFeature,  
    UpdateStationDto,  
    IconPosition8,  
    UchLike,  
    DorLike,  
    NumericLike  
} from './types';  
import { StationDto } from 'services/src/geojsonDashboard.ts';  
import { lineNaprColor, lineUchColor, lineRoadColor } from  
'components/map/src/utils';  
import { Legend } from  
'components/nonScaleMap/components/Legend.tsx';  
import peregonsLocal from './peregons-c.json';  
import { MapSplitModes } from 'store/reducers/src/MapSlice.ts';  
import { ToggleKey } from  
'components/nonScaleMap/components/LayersPopover.tsx';  
import { TopLeftChrome } from  
'components/nonScaleMap/components/TopLeftChrome.tsx';  
import TopRightControls from  
'components/nonScaleMap/components/TopRightControls.tsx';  
import { skipToken } from '@reduxjs/toolkit/query';  
import { AreaType, LocomotiveTractionKind } from 'enums/index.ts';  
import AreaEditor from 'components/areas/AreaEditor.tsx';  
  
// TODO: вынести из этого файла  
const SELECTION_COLOR = 'rgba(40, 194, 40, 1)';  
interface IColoredEdge {  
    edgeId: number;  
    color: string;  
}  
  
const TractionKindToColorMap = {  
    [LocomotiveTractionKind.Diesel]: 'rgb(21, 149, 223)',  
    [LocomotiveTractionKind.Electrical]: 'rgb(209, 212, 9)',  
    [LocomotiveTractionKind.DieselAndElectrical]: 'rgb(20, 224,  
71)'  
};  
  
// eslint-disable-next-line react/display-name  
export const NonScaleMap = forwardRef<GeojsonDashboardHandle,  
GeojsonDashboardProps>(  
    ({ onSidebarChange, onRequestSplit, onRequestSingle,
```



```

onMirrorEvent, indicatorSlot = 1 }, ref) => {
  const dispatch = useAppDispatch();
  const { geoSettings } = useGeojsonDashboardSettings();
  const geoSettingsRef = useRef(geoSettings);

```

Продолжение листинга А.1

```

  const onMirrorEventRef = useRef<typeof
onMirrorEvent>(onMirrorEvent);
  useEffect(() => {
    onMirrorEventRef.current = onMirrorEvent;
  }, [onMirrorEvent]);

  const suppressEmitRef = useRef(false);
  const emit = useCallback((evt: MirrorEvent) => {
    if (!suppressEmitRef.current)
onMirrorEventRef.current?.(evt);
  }, []);

  const pendingViewportRef = useRef<{ pos: { x: number; y:
number }; scale: number } | null>(null);
  const viewportRafRef = useRef<number | null>(null);

  function flushViewport() {
    viewportRafRef.current = null;
    const payload = pendingViewportRef.current;
    pendingViewportRef.current = null;
    if (!payload) return;
    emit({ type: 'viewport', pos: payload.pos, scale:
payload.scale });
  }

  function emitViewport(pos: { x: number; y: number },
scale: number) {
    pendingViewportRef.current = { pos, scale };
    if (!viewportRafRef.current) {
      viewportRafRef.current =
requestAnimationFrame(flushViewport);
    }
  }

  const [editingArea, setEditingArea] = useState<AnyArea |
undefined>();
  const [editingAreaType, setEditingAreaType] =
useState<AreaType>(AreaType.LocomotiveCirculation);
  const [openAreaEditor, setOpenAreaEditor] =
useState<boolean>(false);
  const handleEndSelectionButtonClick = () => {
    setOpenAreaEditor(true);
  };

  const [selectedStationId, setSelectedStationId] =
useState<number | string | null>(null);
  const [netReady, setNetReady] = useState(false);

```

```

    const [selectedStationName, setSelectedStationName] =
useState<string>('');
    const [openSidebar, setOpenSidebar] = useState(false);
    const [stationSettings, setStationSettings] = useState({

```

Продолжение листинга А.1

```

        borderColor: '#000000',
        stationIconSize: geoSettings.stationIconSize,
        fontSize: 7,
        signaturePosition: 0,
        iconType: 'circle' as 'circle' | 'pole' | 'anchor',
        iconPosition: 's' as IconPosition8
    });

    const [nodeDrafts, setNodeDrafts] = useState<
    Record<
        string | number,
        {
            x: number;
            y: number;
            signaturePosition?: number;
            signaturePositionIcons?: number;
            iconType?: number | null;
        }
    >
    >({});
    const {
        data: stationsData,
        error: stationsError,
        isLoading: stationsLoading
    } = stationsAPI.useGetAllStationsQuery();
    console.log(stationsData, 'station');
    const { data: areasApi = [] } =
geojsonDashboardAPI.useGetAreasQuery();
    const [updateStation] =
geojsonDashboardAPI.useUpdateStationMutation();
    const skipAutoFitRef = useRef(false);
    // const { data: peregonsData, error: peregonsError,
isLoading: peregonsLoading } =
    //
geojsonDashboardgetPeregonsCurvesAPI.useGetPeregonsCurvesQuery();
    const peregonsData = useMemo(() => {
        const rows: PeregonRow[] = (peregonsLocal as
any)?.features ?? [];
        return {
            features: rows.map((r) => ({
                properties: {
                    id: Number(r.id),
                    stan0Id: Number(r.stan0_id ?? r.stanA),
                    stan1Id: Number(r.stan1_id ?? r.stanB),
                    name: r.name,
                    dorCode: Number(r.dor_code ?? r.DORCODE ??
r.dorCode),

```

```

        length: r.length != null ?
Number(r.length) : undefined,
        chet: typeof r.chet === 'boolean' ? r.chet
: r.chet != null ? Number(r.chet) === 1 : undefined,

```

Продолжение листинга А.1

```

        trackCount: r.track_count != null ?
Number(r.track_count) : 1,
        coalesce: r.coalesce,
        naprId: r.id_napr ?? r.napr_id ?? r.naprId
?? null,
        uchId: r.id_uch ?? r.uch_id ?? r.uchId ??
null,
        geometry: r.geometry ?? null
    } as PeregonProps
  )))
  };
}, []);
const peregonsLoading = false;
const peregonsError = null;
const { data: stanDocs } =
documentService.useGetStanWithObjectsQuery();
// const { data: peregonsData, error: peregonsError,
isLoading: peregonsLoading } = stationsAPI.useGetPeregonsQuery();

const buildEdgeColorsByPropWithPredicate = (
  prop: keyof PeregonProps,
  lut: Record<string, string>,
  predicate: (props: PeregonProps) => boolean
) => {
  const out: Record<number, string> = {};
  for (const ed of rawEdgesRef.current) {
    const p = ed.properties;
    if (!predicate(p)) continue;
    const v = p[prop];
    const c = lut[String(v)];
    if (c) out[ed.id] = c;
  }
  return out;
};

const handleUpdateStation = (field: keyof
StationSettingsState, value: any) => {
  const newSettings = { ...stationSettings, [field]:
value };
  setStationSettings(newSettings);

  if (!nodesDataSetRef.current || selectedStationId ==
null) return;

  const patch: NodeUpdatePayload = { id:
selectedStationId! };

```

```

switch (field) {
  case 'fontSize':
    patch.customFontSize = newSettings.fontSize;
    break;

```

Продолжение листинга А.1

```

    case 'signaturePosition':
      patch.signaturePosition =
newSettings.signaturePosition;
      break;

    case 'borderColor':
    case 'stationIconSize': {
      const size = newSettings.stationIconSize;
      const border = newSettings.borderColor;
      patch.stationIconSize = size;
      patch.size = size;
      patch.borderColor = border;
      patch.image = createCircleSvgDataURL(border,
2, '#ffffff', size);
      break;
    }

    case 'iconType': {
      patch.iconType = newSettings.iconType;
      patch.iconImage =
        newSettings.iconType === 'pole'
          ? PoleSvg
          : newSettings.iconType === 'anchor'
            ? AnchorSvg
            : createCircleSvgDataURL(
                newSettings.borderColor,
                2,
                '#ffffff',
                newSettings.stationIconSize
              );

      const num =
ICON_TYPE_TO_NUM[newSettings.iconType];
      setNodeDrafts((prev) => ({
        ...prev,
        [selectedStationId!]: {
          ...(prev[selectedStationId!] || {
            x:
allNodePositionsRef.current[selectedStationId!].x,
            y:
allNodePositionsRef.current[selectedStationId!].y
          }),
          iconType: num
        }
      }));
      break;

```

```

    }

    case 'iconPosition': {
      const pos = newSettings.iconPosition as

```

Продолжение листинга A.1

```

IconPosition8;

      const posNum = ICON_POS8_TO_NUM[pos];
      patch.iconPosition = pos;
      patch.signaturePositionIcons = posNum;

      setNodeDrafts((prev) => ({
        ...prev,
        [selectedStationId!]: {
          ...(prev[selectedStationId!] || {
            x:
allNodePositionsRef.current[selectedStationId!].x,
            y:
allNodePositionsRef.current[selectedStationId!].y
          }),
          signaturePositionIcons: posNum
        }
      }));
      break;
    }
  }

  nodesDataSetRef.current.update(patch);

  emit({
    type: 'nodeSettings',
    id: selectedStationId!,
    changes: (() => {
      switch (field) {
        case 'fontSize':
          return { fontSize:
newSettings.fontSize };
        case 'signaturePosition':
          return { signaturePosition:
newSettings.signaturePosition };
        case 'borderColor':
        case 'stationIconSize':
          return {
            borderColor:
newSettings.borderColor,
            stationIconSize:
newSettings.stationIconSize
          };
        case 'iconType':
          return { iconType:
newSettings.iconType };
        case 'iconPosition':
          return { iconPosition:

```

```

newSettings.iconPosition };
                        default:
                            return {};
                    }

```

Окончание листинга A.1

```

                }) ()
            });

            nodesViewRef.current?.refresh();
            networkRef.current?.redraw();
        };

        case 'iconType':
            return { iconType: newSettings.iconType };
        case 'iconPosition':
            return { iconPosition: newSettings.iconPosition };
            return {};
        }
    }) ()
});    default:

```

